

**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE ENSENADA
INGENIERIA EN SISTEMAS COMPUTACIONALES**

AUTOMATIZACIÓN DE PROCESOS E IND. 4.0

**SISTEMA DIGITAL PARAMONITOREO DE COLMENAS BASADO EN
INDUSTRIA 4.0**

ELABORADO POR:

GATICA MARTINEZ YOSSY LEYDI	21760237
DANIEL ISAAC GARCIA NUÑEZ	22760532

CATEDRÁTICO: GUILLERMO ALEJANDRO CHÁVEZ

ENSENADA B.C. A 26 DE MAYO DE 2026

1. INTRODUCCIÓN

La apicultura constituye una actividad fundamental para el equilibrio ecológico y la producción agrícola debido a la importancia de las abejas en los procesos de polinización. Actualmente, las colmenas enfrentan diversos problemas relacionados con enfermedades, cambios climáticos, disminución de población y reducción en la producción de miel.

Muchos de estos problemas suelen detectarse tardíamente debido a que el monitoreo tradicional se realiza mediante inspecciones físicas y manuales que alteran el ambiente interno de la colmena. Con el avance de las tecnologías asociadas a la Industria 4.0 y el Internet de las Cosas (IoT), es posible implementar sistemas inteligentes capaces de supervisar variables importantes dentro de una colmena utilizando sensores electrónicos, conectividad inalámbrica y almacenamiento de datos en la nube.

El presente proyecto propone el diseño e implementación de un sistema IoT inteligente para el monitoreo de colmenas utilizando el microcontrolador ESP32, sensores de temperatura, peso y movilidad, permitiendo supervisar en tiempo real el comportamiento general de la colmena y generar alertas automáticas ante posibles anomalías.

El sistema busca automatizar la recopilación de información, disminuir inspecciones invasivas y proporcionar datos útiles para mejorar la toma de decisiones dentro del sector apícola.

Asimismo, el proyecto permite aplicar conocimientos relacionados con electrónica, programación, redes, automatización y plataformas IoT, integrando tecnologías propias de la Industria 4.0 en un entorno académico y tecnológico.

2. OBJETIVO GENERAL Y ESPECÍFICOS

Objetivo General

Diseñar e implementar un sistema IoT para el monitoreo inteligente de colmenas mediante sensores electrónicos, conectividad inalámbrica y almacenamiento en la nube, con la finalidad de supervisar variables relacionadas con el estado y comportamiento de una colmena en tiempo real.

Objetivos Específicos

- Implementar sensores capaces de medir temperatura, peso y movilidad en la colmena.
- Utilizar el microcontrolador ESP32 como unidad principal de procesamiento.
- Desarrollar comunicación inalámbrica mediante conexión WiFi.
- Enviar información a una plataforma de almacenamiento en la nube.
- Visualizar datos en tiempo real mediante un Dashboard de monitoreo.
- Reducir inspecciones físicas invasivas dentro de la colmena.
- Generar registros históricos para análisis y seguimiento del comportamiento del panel.
- Implementar alertas automáticas ante cambios bruscos en las variables monitoreadas.

3. MARCO TEÓRICO

Industria 4.0

La Industria 4.0 representa la cuarta revolución industrial basada en la automatización, digitalización e interconectividad de dispositivos inteligentes. Este modelo tecnológico permite recopilar y analizar información en tiempo real mediante sensores, internet y plataformas digitales. En este proyecto se aplican principios de Industria 4.0 mediante la integración de sensores inteligentes, conectividad WiFi y monitoreo remoto.

Internet de las Cosas (IoT)

El Internet de las Cosas (IoT) consiste en la conexión de dispositivos físicos a internet para recopilar, enviar y procesar información automáticamente.

El sistema desarrollado utiliza sensores conectados al ESP32 para obtener información relacionada con la colmena y transmitirla hacia un servidor mediante conexión inalámbrica.

ESP32

El ESP32 es un microcontrolador de bajo costo que incorpora conectividad WiFi y Bluetooth, siendo ampliamente utilizado en proyectos IoT debido a su capacidad de procesamiento y facilidad de integración con sensores electrónicos.

Dentro del proyecto, el ESP32 funciona como el cerebro principal del sistema encargado de:

- Leer sensores
- Capturar datos
- Conectarse a internet
- Enviar información al servidor

Sensor BME280

El sensor BME280 permite medir variables ambientales como:

- Temperatura
- Humedad
- Presión atmosférica

En el proyecto se utiliza principalmente para supervisar la temperatura dentro de la colmena, ya que las abejas requieren condiciones térmicas relativamente estables para mantener un adecuado funcionamiento.

Sistema de Peso HX711

El sistema de peso se compone de una celda de carga y un módulo HX711 encargado de amplificar y convertir señales analógicas provenientes del sensor de peso.

Este sistema permite monitorear variaciones de peso relacionadas con:

Producción de miel

Cambios poblacionales

Actividad de la colmena

Sensor de Movilidad

El proyecto considera el uso de sensores de movilidad para detectar actividad cercana a la entrada de la colmena, permitiendo analizar patrones básicos de comportamiento y actividad de las abejas.

Firestore

Firestore es una plataforma de almacenamiento en la nube utilizada para guardar y visualizar datos en tiempo real.

El sistema envía información hacia Firestore mediante conexión WiFi utilizando peticiones HTTP en formato JSON.

4. ARQUITECTURA DEL SISTEMA

La arquitectura general del sistema se divide en cinco etapas principales:

Captura de Datos

Los sensores obtienen información relacionada con temperatura, peso y movilidad dentro de la colmena.

Procesamiento

El ESP32 procesa la información recibida desde los sensores y ejecuta la lógica de envío de datos.

Comunicación

El microcontrolador se conecta a internet mediante WiFi para transmitir los datos recopilados.

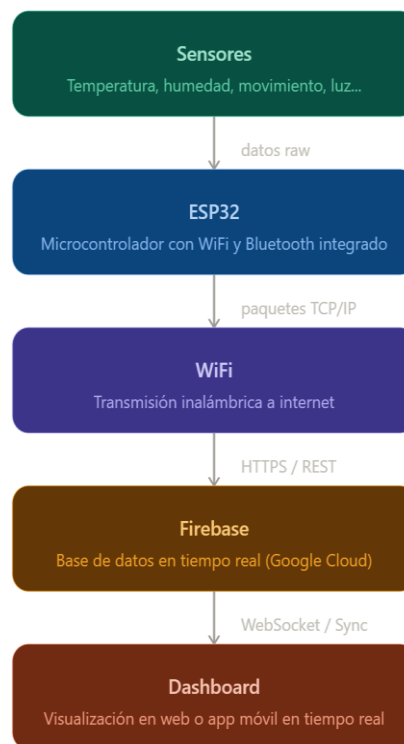
Almacenamiento

La información es enviada y almacenada en una base de datos en la nube.

Visualización

Los datos pueden visualizarse mediante Dashboard o aplicaciones de monitoreo en tiempo real.

Flujo General del Sistema:



Toca cada bloque para saber más

5. COMPONENTES HARDWARE

Componente	Función
ESP32	Procesamiento y conectividad
BME280	Medición de temperatura
HX711	Amplificación de señal de peso
Celda de carga	Medición de peso
Sensor de movilidad	Detección de actividad
Protoboard	Pruebas electrónicas
Cables jumper	Conexiones
Cable USB	Alimentación y programación

6. SOFTWARE Y CONFIGURACIÓN

Software	Función
Arduino IDE	Programación del ESP32
Firebase	Almacenamiento en nube

Configuración del ESP32

1. Instalar Arduino IDE.
2. Instalar soporte para ESP32.
3. Seleccionar la tarjeta ESP32 Dev Module.
4. Conectar el ESP32 mediante cable USB.
5. Configurar red WiFi y servidor dentro del archivo config.h

Configuración WiFi

El ESP32 requiere conexión a una red inalámbrica para transmitir datos hacia la nube.

7. INSTALACIÓN Y PUESTA EN MARCHA

Paso 1. Montaje Electrónico

Se conectan los sensores al ESP32 utilizando protoboard y cables jumper.

Paso 2. Alimentación

El sistema se energiza mediante cable USB conectado al ESP32.

Paso 3. Programación

El código fuente se carga desde Arduino IDE hacia el ESP32.

Paso 4. Verificación de Sensores

Se revisa el funcionamiento correcto de:

Sensor BME280

Sistema de peso HX711

Sensores de movilidad

Paso 5. Conexión a Internet

El ESP32 se conecta automáticamente a la red WiFi configurada.

Paso 6. Envío de Datos

Los datos son transmitidos hacia Firebase o servidor local mediante peticiones HTTP.

8. MODO DE EMPLEO

1. Encender el sistema mediante conexión USB.
2. Verificar conexión WiFi.
3. Confirmar funcionamiento de sensores.
4. Supervisar información desde el dashboard.
5. Consultar registros históricos almacenados.
6. Revisar alertas automáticas generadas por el sistema.

9. SOLUCIÓN DE PROBLEMAS

Problema	Posible causa	Solución
ESP32 no conecta a WiFi	Credenciales incorrectas	Revisar usuario y contraseña
Sensor no responde	Conexiones incorrectas	Revisar cableado
Lecturas erróneas	Mala calibración	Recalibrar sensores
Reinicios del sistema	Alimentación insuficiente	Revisar fuente de energía
Firebase no recibe datos	Error de configuración	Revisar URL y conexión

7. MANTENIMIENTO

- Para garantizar el correcto funcionamiento del sistema se recomienda:
- Revisar periódicamente conexiones eléctricas.
- Limpiar sensores y protoboard.
- Verificar estabilidad de la red WiFi.
- Calibrar periódicamente el sistema de peso.
- Revisar integridad del cableado.
- Actualizar el software cuando sea necesario.

8. CONCLUSIONES

El desarrollo del sistema IoT para monitoreo inteligente de colmenas permitió integrar conocimientos relacionados con electrónica, programación, automatización y conectividad inalámbrica dentro de un entorno tecnológico basado en Industria 4.0.

El proyecto demuestra que es posible supervisar variables importantes dentro de una colmena utilizando sensores electrónicos y tecnologías IoT, permitiendo reducir inspecciones físicas y facilitando el monitoreo remoto en tiempo real.

El uso del ESP32 permitió integrar sensores, conexión WiFi y comunicación con plataformas en la nube de forma eficiente y económica.

Asimismo, el sistema genera información útil para el análisis del comportamiento de la colmena, contribuyendo al desarrollo de soluciones tecnológicas aplicadas al sector apícola.

9. REFERENCIAS BIBLIOGRÁFICAS

Bravo, F. (n.d.). Regresiones, Modelos lineales y redes neuronales.
<https://felipebravom.com/teaching/regresion.pdf>

Jesús Tomás, V. C. (n.d.). Firebase: trabajar en la nube. Alfaomega.
Ortiz Arcienega, V. B. (n.d.). ESP32: Manual Básico para estudiantes.
(Bravo) (Jesús Tomás) (Ortiz Arcienega)

¿Qué es la industria 4.0? (2026, enero 13). Ibm.com. <https://www.ibm.com/mx-es/think/topics/industry-4-0>

(S/f). Uelectronics.com. Recuperado el 22 de mayo de 2026, de
<https://uelectronics.com/producto/sensor-de-presion-atmosferica-bme280-3-3/>

Carranza, S. (2021, octubre 27). CONOCIENDO AL ESP32. TodoMaker.
<https://todomaker.com/blog/conociendo-al-esp32/>

¿Qué es el internet de las cosas (IoT)? (2026, mayo 8). Ibm.com. <https://www.ibm.com/mx-es/think/topics/internet-of-things>

13. ANEXOS

Arduino IDE para el funcionamiento del sistema IoT de monitoreo inteligente de colmenas. El programa permite:

- Leer sensores de temperatura.
- Simular y procesar datos de peso.
- Detectar actividad o movimiento.
- Conectarse a internet mediante WiFi.
- Enviar datos hacia un servidor utilizando peticiones HTTP.
- Generar alertas automáticas basadas en condiciones del sistema.

El código fue implementado sobre un microcontrolador ESP32 debido a su capacidad de procesamiento y conectividad inalámbrica integrada.

```
/*
  PANAL industria 4.0
  Envío cada 10 min O ante cambios bruscos de peso (>0.4 kg)
  Datos: Peso, Temperatura, Movimiento → Guarda en laptop vía WiFi
*/
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <time.h>
#include <Adafruit_BME280.h>
#include <Adafruit_Sensor.h>
#include "HX711.h"
#include "config.h"

// =====
// CONFIGURACIÓN DE PINES Y SENSORES
// =====
// PIN 25 (GPIO25) → HX711 SCK - Sensor de PESO (Serial Clock)
// PIN 35 (GPIO35) → HX711 DT - Sensor de PESO (Data)
// PIN 21 (GPIO21) → BME280 SDA - Sensor TEMPERATURA/HUMEDAD/PRESIÓN (I2C Data)
// PIN 22 (GPIO22) → BME280 SCL - Sensor TEMPERATURA/HUMEDAD/PRESIÓN (I2C Clock)
// =====

// ===== CONFIGURACIÓN WIFI =====
const char* ssid = WIFI_SSID;
const char* password = WIFI_PASSWORD;
const char* serverURL = SERVER_URL;

// ENCODER ROTATIVO — Simulación de sensor de peso
const int POT_PIN = 34; // GPIO34: lectura analógica del potenciómetro
const float ALFA = 0.10;

// =====
// SENSOR DE PESO HX711
// Lee el peso usando celdas de carga amplificadas
// =====
```

```
const int HX711_DT = 35; // Pin de datos (GPIO35)
const int HX711_SCK = 25; // Pin de reloj (GPIO25)
const float CALIBRATION_FACTOR = -450.0; // Ajusta este valor calibrando tu escala
```

```
// -----
// SENSOR AMBIENTAL BME280 (I2C)
// Lee: Temperatura (°C), Humedad (%), Presión (hPa)
// Protocolo: I2C (comunicación de dos líneas)
// Dirección I2C: 0x76
// -----
```

```
const int SDA_PIN = 21; // I2C Data/SDA (GPIO21)
const int SCL_PIN = 22; // I2C Clock/SCL (GPIO22)
```

```
// LÓGICA DE ENVÍO
// CAMBIO PARA PRESENTACIÓN:
const unsigned long INTERVALO_MS = 15000; // 15 segundos
const float UMBRAL_CAMBIO = 1.0; // kg para disparo inmediato
```

```
float pesoFiltrado = 0.0;
float ultimoPesoEnviado = -1.0;
unsigned long ultimoEnvio = 0;
int recordID = 1;
```

```
// BME280 Sensor
Adafruit_BME280 bme;
bool bmeSensor = false;
```

```
// HX711 Sensor de Peso
HX711 scale;
bool weightSensor = false;
```

```
//
```

```
=====
// ESTRUCTURAS DE DATOS PARA SENSORES
//
```

```
=====
struct SensorPeso {
    float pesoRaw;
    float pesoFiltrado;
    float pesoMinimo;
    float pesoMaximo;
    bool disponible;
    uint32_t ultimaLectura;
};
```

```
struct SensorAmbiente {
    float temperatura;
    float humedad;
    float presion;
    bool disponible;
    uint32_t ultimaLectura;
};
```

```
struct DatosColmena {
```

```
uint32_t id;
char timestamp[30];
SensorPeso peso;
SensorAmbiente ambiente;
int movimiento;
char alerta[30];
int httpCode;
char motivo[20];
};

struct BME280Data {
    float temperatura;
    float humedad;
    float presion;
};

void setup() {
    Serial.begin(115200);
    delay(1000);

    // WiFi
    WiFi.begin(ssid, password);
    Serial.print("Conectando");
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nWiFi OK | IP ESP32: " + WiFi.localIP().toString());

    // Hora real
    configTime(-6 * 3600, 0, "pool.ntp.org", "time.nist.gov");

    // Configurar pines I2C para BME280
    Wire.begin(SDA_PIN, SCL_PIN);

    // =====
    // INICIALIZAR SENSOR BME280 (TEMPERATURA/HUMEDAD/PRESIÓN)
    // Pin: 21 (SDA) y 22 (SCL) - Protocolo: I2C
    // =====
    // BME280 Inicialización
    Serial.print("[INIT] Intentando conectar con BME280 en dirección 0x76 (I2C: SDA=21, SCL=22)...");
    if (!bme.begin(0x76)) {
        Serial.println(" FALLO");
        Serial.println("[ERROR] BME280 NO encontrado. Continuando sin sensor ambiental.");
        bmeSensor = false;
    } else {
        Serial.println(" ✓ OK");
        Serial.println("[INFO] BME280 inicializado correctamente!");
        bmeSensor = true;
    }
}

// =====
// SENSOR DE PESO — Potenciometro en GPIO 34 (simulación)
// Rango: 0.0 – 60.0 kg
// =====
Serial.print("[INIT] Configurando potenciometro en GPIO 34 (simulación de peso)...");
pinMode(POT_PIN, INPUT);
weightSensor = true;
Serial.println(" ✓ OK");
Serial.println("[INFO] Rango configurado: 0.0 - 60.0 kg");
```

```
// ADC
analogSetAttenuation(ADC_11db);
Serial.println("[SUCCESS] Sistema listo. Esperando datos...\n");
}

// Lectura del sensor BME280
SensorAmbiente leerBME280() {
  SensorAmbiente datos;
  datos.ultimaLectura = millis();

  if (bmeSensor) {
    datos.temperatura = bme.readTemperature();
    datos.humedad = bme.readHumidity();
    datos.presion = bme.readPressure() / 100.0; // Convertir a hPa
    datos.disponible = true;
    Serial.printf("[BME280] Temp: %.2f°C | Humedad: %.1f%% | Presión: %.2f hPa\n",
      datos.temperatura, datos.humedad, datos.presion);
  } else {
    // Valores por defecto si sensor no disponible
    datos.temperatura = 24.0;
    datos.humedad = 50.0;
    datos.presion = 1013.25;
    datos.disponible = false;
    Serial.println("[BME280] Sensor no disponible. Usando valores por defecto.");
  }
  return datos;
}

// Lectura de peso via potenciómetro en GPIO 34
SensorPeso leerHX711() {
  SensorPeso datos;
  datos.ultimaLectura = millis();

  // Promediar 10 lecturas ADC para reducir ruido
  long suma = 0;
  for (int i = 0; i < 25; i++) {
    suma += analogRead(POT_PIN);
    delay(2);
  }
  float rawADC = suma / 10.0;

  // Mapear ADC (0-4095) a rango de peso (0-60 kg)
  datos.pesoRaw = (rawADC / 4095.0) * 60.0;
  datos.disponible = true;

  // Aplicar filtro exponencial
  pesoFiltrado = (pesoFiltrado * (1.0 - ALFA)) + (datos.pesoRaw * ALFA);

  // Descartar valores muy pequeños (ruido)
  if (pesoFiltrado < 0.3) pesoFiltrado = 0.0;

  // Limitar rango (0-60 kg)
  datos.pesoFiltrado = constrain(pesoFiltrado, 0.0, 60.0);

  // Registrar mínimo y máximo
  if (datos.pesoFiltrado > 0) {
```

```
datos.pesoMinimo = (datos.pesoMinimo == 0) ? datos.pesoFiltrado : min(datos.pesoMinimo, datos.pesoFiltrado);
datos.pesoMaximo = max(datos.pesoMaximo, datos.pesoFiltrado);
}

Serial.printf("[PESO] ADC: %.0f | Raw: %.3f kg | Filtrado: %.3f kg | Min: %.3f kg | Max: %.3f kg\n",
    rawADC, datos.pesoRaw, datos.pesoFiltrado, datos.pesoMinimo, datos.pesoMaximo);
return datos;
}

void loop() {
    Serial.println("\n===== CICLO NUEVO =====");
    struct tm timeinfo;
    if (!getLocalTime(&timeinfo)) {
        Serial.println("[ERROR] No se pudo obtener la hora del NTP. Reintentando...");
        delay(1000);
        return;
    }

    // LEER TODOS LOS SENSORES Y GUARDAR EN ESTRUCTURAS
    SensorPeso pesoData = leerHX711();
    float pesoActual = pesoData.pesoFiltrado;

    // Lectura del sensor BME280
    SensorAmbiente ambienteData = leerBME280();
    float temp = ambienteData.temperatura;
    float humidity = ambienteData.humedad;
    float pressure = ambienteData.presion;

    int mov = (pesoActual > 2.0 && pesoActual < 8.0) ? 1 : 0;
    Serial.printf("[MOVIMIENTO] %s\n", mov ? "Actividad detectada" : "Sin movimiento");

    bool enviar = false;
    String motivo = "";

    // Condición 1: Cada 10 minutos
    if (ultimoEnvio == 0 || millis() - ultimoEnvio >= INTERVALO_MS) {
        enviar = true;
        motivo = "Intervalo 10 min";
        Serial.printf("[TRIGGER] Envío por intervalo (Pasaron %lu ms)\n", millis() - ultimoEnvio);
    }

    // Condición 2: Cambio brusco de peso
    else if (ultimoPesoEnviado >= 0 && abs(pesoActual - ultimoPesoEnviado) >= UMBRAL_CAMBIO) {
        enviar = true;
        motivo = "Cambio brusco";
        Serial.printf("[TRIGGER] Cambio brusco detectado (%.3f kg → %.3f kg, diferencia: %.3f kg)\n",
            ultimoPesoEnviado, pesoActual, abs(pesoActual - ultimoPesoEnviado));
    }
    else {
        Serial.println("[INFO] Sin condiciones para envío. Esperando...");
    }

    if (enviar) {
        Serial.println("[SEND] ===== ENVIANDO DATOS =====");
        char ts[30];
        strftime(ts, sizeof(ts), "%Y-%m-%dT%H:%M:%S", &timeinfo);
        Serial.printf("[SEND] Timestamp: %s\n", ts);

        String alerta = "OK";
```

```

if (pesoActual > 9.5) {
    alerta = "Cosecha (>9.5kg)";
    Serial.println("[ALERTA] COSECHA: Peso superior a 9.5kg");
}
else if (pesoActual < 4.0) {
    alerta = "Inanición (<4.0kg)";
    Serial.println("[ALERTA] INANICIÓN: Peso inferior a 4.0kg");
}

HTTPClient http;
WiFiClientSecure client;
client.setInsecure(); // Cloudflare tunnel - sin verificación de certificado
String url = String(serverURL) + "/data";
Serial.printf("[SEND] URL: %s\n", url.c_str());
http.begin(client, url);
http.addHeader("Content-Type", "application/json");

String json = "{"id\":" + String(recordID++) +
    "\",\"timestamp\":" + String(ts) +
    "\",\"peso\":" + String(pesoActual, 3) +
    "\",\"temperatura\":" + String(temp, 2) +
    "\",\"humedad\":" + String(humidity, 1) +
    "\",\"presion\":" + String(pressure, 2) +
    "\",\"movimiento\":" + String(mov) +
    "\",\"alerta\":" + alerta + "}";
Serial.println("[SEND] Enviando petición HTTP POST...");
Serial.printf("[SEND] JSON: %s\n", json.c_str());

int code = http.POST(json);

if (code == 200) {
    Serial.printf("[SUCCESS] HTTP %d - Datos enviados correctamente!\n", code);
} else {
    Serial.printf("[ERROR] HTTP %d - Error en la petición\n", code);
}

// LÍNEA COMPLETA CON TODOS LOS DATOS:
Serial.printf("\n[RESUMEN] [%s] %s | Peso: %.3f kg | Temp: %.2f °C | Humedad: %.1f %% | Presión: %.2f hPa | Mov: %d |
Alerta: %s | HTTP: %d\n",
    ts, motivo.c_str(), pesoActual, temp, humidity, pressure, mov, alerta.c_str(), code);
http.end();

ultimoEnvio = millis();
ultimoPesoEnviado = pesoActual;
Serial.println("[SEND] Pausa técnica de 2 segundos post-envío...");
Serial.println("=====\n");
delay(2000); // Pausa técnica post-envío
}

Serial.println("[LOOP] Próximo ciclo en 1 segundo...\n");
delay(1000); // Ciclo principal
}

```

El sistema electrónico integra:

- Arduino ESP32- Conectividad WIFI y Bluetooth
- Sensor BME280 – temperatura, humedad y presión hidrostática.
- Celda de carga y transmisor de peso HX711

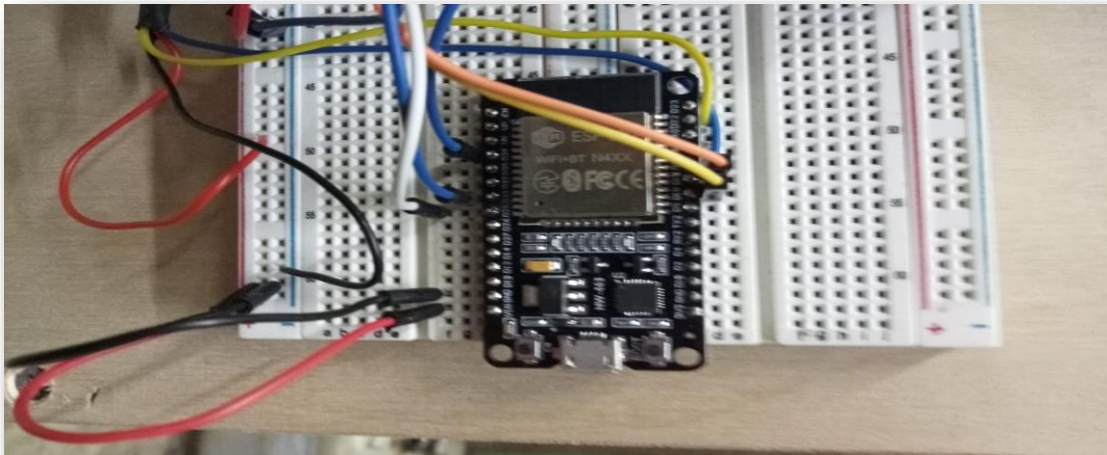


Figure 1 En la imagen se observa el montaje principal del sistema utilizando un microcontrolador ESP32 instalado sobre una protoboard para facilitar las conexiones electrónicas y pruebas del prototipo. El ESP32 funciona como la unidad central de procesamiento encargada de leer los sensores, procesar los datos y realizar la comunicación inalámbrica mediante WiFi.

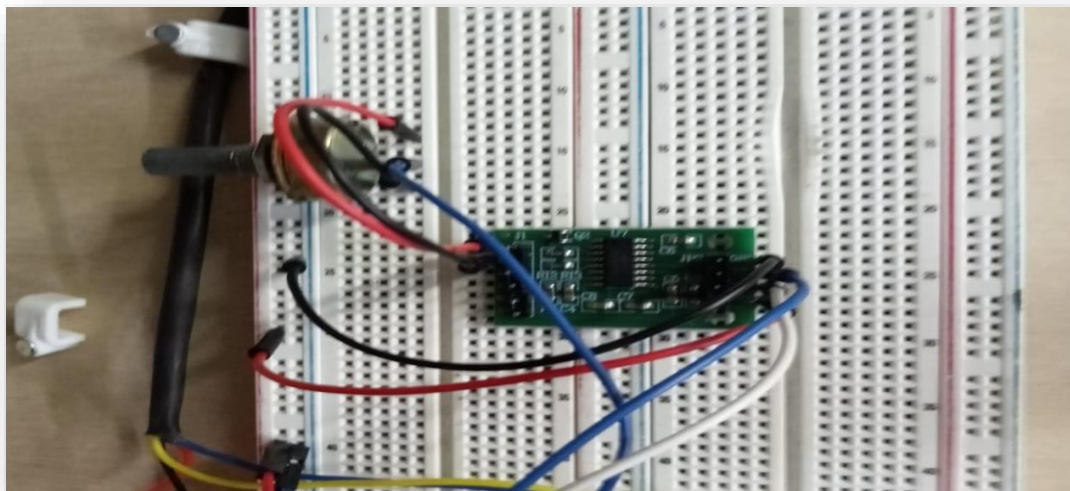


Figure 2 La imagen muestra el módulo HX711 utilizado para la adquisición y amplificación de señales provenientes del sistema de peso. Durante las pruebas iniciales del proyecto se utilizó un potenciómetro como simulador de variaciones de peso para validar el funcionamiento del sistema antes de integrar una celda de carga real.

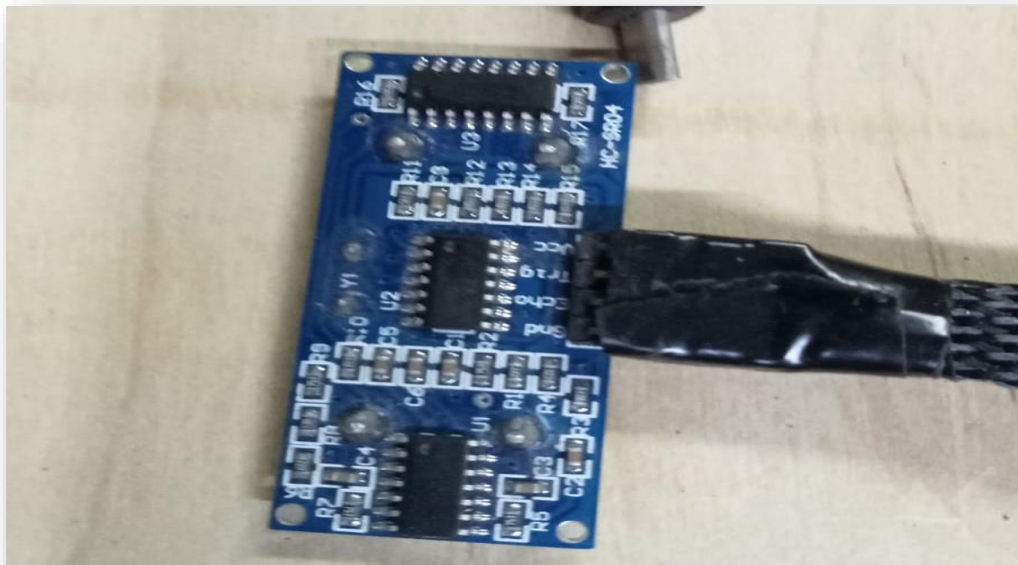


Figure 3 En la imagen se observa un sensor digital de temperatura tipo DS18B20 utilizado para medir las condiciones térmicas dentro de la colmena. Este sensor permite obtener lecturas precisas de temperatura y transmitir las digitalmente hacia el ESP32.